StockSense

Smart inventory and reorder assistant for small businesses

Local prototype MVP

Software Design I / Fall 2026

Raaj Patel

Purdue University Northwest

Instructor: OT Ajayi

Prepared October 28, 2026

Repository: https://github.com/Raajp10/stocksense

This report describes the implemented application and the verification evidence available at preparation time. The café records, sales, suppliers and product illustrations are fictional demonstration fixtures. Business benefits are intended outcomes rather than measured results from a real deployment.

The required release uses Python 3.12, Flask, Jinja2, SQLAlchemy, SQLite/PostgreSQL, Bootstrap, Chart.js, pytest and Playwright. It includes QR-based product lookup, location balances, shelf refill suggestions and explainable supplier reorder recommendations.

Implementation and report drafting received significant assistance from OpenAI Codex. The student is responsible for reviewing, understanding and defending the delivered work. Details are recorded in docs/ai-usage.md.

Contents and reading guide

Chapter 1 - Introduction: background, motivation and objectives.

Chapter 2 - Problem Definition and Requirements Analysis: users, functional scope, quality constraints and acceptance criteria.

Chapter 3 - Software Design and Architecture: component diagram, entity model, transactional consistency and calendar-based calculations.

Chapter 4 - Implementation: catalog and media, QR workflows, inventory services, purchase orders, dashboard and exports.

Chapter 5 - Testing and Results: automated evidence, concurrency and rollback testing, interface screenshots, installation and release verification.

Chapter 6 - Conclusion, Limitations, and Future Improvements.

References - consistent numbered source references.

The report is accompanied by runnable source, a dependency lock, migration history, a development seed, tests, a root README and supporting design documents. The specification is preserved in PLAN.md. docs/requirements.md maps the sixteen core requirements to implementation and verification. TASKS.md distinguishes implemented work from checks that still require external services or the student's demonstration device.

The course guidelines require an executable application, public GitHub repository, README and report of at least twenty pages [1]. The approved proposal supplies the baseline problem, users and technologies [2]. QR labels, photos and location tracking extend that proposal following the student's supplied follow-up requirements. Instructor approval of the proposal does not independently certify these additions or the resulting implementation.

Chapter 1: Introduction

1.1 Background and motivation

A small café must know what remains on the shelf, what is available in the backroom and what should be purchased next. A notebook or spreadsheet can capture quantities, but its numbers become difficult to trust when several people record sales and deliveries. An item may appear unavailable at the sales shelf even when a reserve exists elsewhere in the store. Conversely, a full-looking shelf may conceal insufficient total stock for the supplier's lead time.

StockSense addresses these operational questions with an inventory application built around explicit movements. Staff record sales from a physical source, deliveries to a destination and internal transfers between locations. Each action contributes to a visible history. Owners configure products, supplier lead times, thresholds and shelf targets. A deterministic calculation turns recent recorded sales into an explainable purchasing suggestion.

The proposal identifies small retailers, cafés, restaurants, campus stores and similar businesses as intended users [2]. This implementation uses a café as its demonstration setting and limits inventory to whole-number retail units. It does not claim to manage ingredient recipes, food-safety controls, lot expiry or payment processing. Those boundaries keep the project focused on traceable stock arithmetic and a reliable classroom demonstration.

The motivation is practical visibility: a person should be able to find an item, see its quantity at each location and understand the next recommended action. Product descriptions and photos support human identification. QR labels reduce the need to search by name, while a manual SKU path ensures the workflow remains usable when a camera is unavailable.

1.2 Intended value

The intended benefits are less manual searching, clearer purchasing decisions and fewer mistakes from repeated submissions. No real business study was conducted. Therefore, reductions in waste, stockouts or operating cost remain expected benefits, not measured outcomes. The report evaluates software behavior against defined test cases instead.

Chapter 1: Introduction

1.3 Objectives and success criteria

The first objective is a consistent inventory model. Product stock must equal the sum of its location balances. A sale decreases only its selected source. A receipt increases only its selected destination. A transfer moves the same quantity between two locations and leaves the product total unchanged. Insufficient source stock must produce a clear rejection even when a different location contains available units.

The second objective is accountable operation. Every successful stock change must have a movement record with an actor, location, quantity, reason or reference and operation identifier. An owner-approved correction must refuse to overwrite a physical count when the location changed after the count was submitted. Historical sales must retain the price charged at recording time.

The third objective is explainable replenishment. The application uses the previous fourteen completed business-calendar days, includes zero-sale days and excludes the current incomplete day. Supplier lead time plus safety days determines coverage. The user sees the forecast target, manual threshold, stock and approved incoming units rather than an unexplained recommendation.

The fourth objective is an executable, reviewable handoff. A new environment should install locked dependencies, migrate an empty database, seed fictional data and launch the application using the README. Tests must cover arithmetic, role boundaries, duplicate requests, concurrent writes and rollback failures. Screenshots must show the running application rather than an imagined interface.

These objectives are intentionally evaluated separately from hosting and the classroom presentation. A Render configuration does not prove a deployed service. Synthetic video decoding does not prove physical camera performance. A student must still rehearse the actual device and explain the code during the December 9 demonstration [1]. The release checklist preserves those distinctions.

Chapter 2: Problem Definition and Requirements Analysis

2.1 Users, operating context and assumptions

The owner is responsible for catalog data, supplier configuration, purchase decisions and staff access. Owners can create and edit products, set costs and selling prices, upload images, manage locations, configure shelf targets, approve corrections and approve or cancel purchase orders. Cost-based dashboard metrics and exports are restricted to owners.

Staff members perform daily stock work. They can view inventory, find products by QR or SKU, record sales and standalone deliveries, transfer stock, receive an approved complete order and submit a physical count. They cannot change product prices, manage supplier settings or approve purchase orders. These restrictions apply in server handlers and services; hiding interface buttons alone is insufficient.

Each user belongs to one store. Products, suppliers, locations, balances and transactions carry store identifiers. A second fixture store supports isolation tests. The same SKU can exist in two stores, but a signed-in user can only resolve the item belonging to their store. Submitted store identifiers are never accepted as a replacement for the authenticated account's store.

The application assumes whole-number units such as cartons, bags, cups or individually stocked boxes. A box and an individual unit require separate SKUs if they are stocked independently. Scanning a box never silently converts it into loose items. Locations represent shelves, bins or a backroom within one store, rather than geographically distinct branches.

The owner supplies initial records and staff accounts; there is no public registration. Sales describe inventory movements and revenue snapshots rather than payment transactions. Supplier information is manually maintained. The core workflow runs without an AI API or supplier network connection, supporting a reliable local classroom demonstration.

Chapter 2: Problem Definition and Requirements Analysis

2.2 Functional requirements

FR01 requires hashed-password owner/staff sign-in, logout and authenticated routes. FR02 requires an owner-managed catalog with SKU, category, unit, supplier, prices and reorder level. FR03 requires supplier contact details, lead time and safety days. FR04 requires atomic sales and deliveries with matching movement history.

FR05 requires physical-count submission and owner approval, including protection against stale counts. FR06 requires manual-threshold and forecast-risk warnings. FR07 requires a visible, sales-based reorder calculation. FR08 requires supplier-grouped drafts, editable quantities and destinations, approval, cancellation, CSV export and complete receipt once.

FR09 requires dashboard valuation, sales activity and fastest/slowest products. FR10 requires owner/store-scoped inventory, sales and reorder exports, including date ranges where applicable. FR11 requires documented installation, migration, seeding, tests and execution from a fresh source copy.

The expanded requirements add product photos and descriptions (FR12), stable printable QR labels and scanning (FR13), per-location quantities and filtering (FR14), atomic internal stock transfers (FR15), and source-limited shelf refill suggestions (FR16). A QR identifies a product type; a scan itself must never change quantity. The user must select a positive whole-number quantity and appropriate locations, review the projected change and explicitly confirm.

Supplier purchasing and shelf refill remain separate decisions. An empty shelf can need an internal transfer without requiring a supplier order. Conversely, transferring existing stock cannot increase the total available to the business. The interface uses different pages and language for these actions to preserve that distinction.

The plan also names optional AI questions, supplier email, PDF order export and other future features. They are not required for the core release. The delivered implementation uses CSV orders and deterministic recommendations and does not represent those stretch goals as completed capabilities.

Chapter 2: Problem Definition and Requirements Analysis

2.3 Quality requirements and acceptance scenario

Correctness takes priority over feature count. Nonnegative balances, store-consistent references, positive movement quantities and valid lifecycle states are enforced through validation and database constraints. The stock service must commit the balance and its ledger together. A failed second transfer leg or purchase-order line must roll back the entire action.

Usability requirements include labelled controls, readable feedback, empty states, phone-width layouts and preserved field values after validation errors. A disabled submit button helps prevent accidental double clicks, but the server must independently enforce duplicate protection. Success appears only after a committed server response. Camera denial must leave manual SKU entry available.

Security requirements include password hashing, CSRF protection on mutations, owner authorization, escaped descriptions, a production secret, safe session cookies and bounded image decoding. Uploaded media accepts decoded JPEG/PNG/WebP content, normalizes it to a bounded JPEG and stores it under a random key. Arbitrary SVG or HTML uploads are rejected. Camera frames remain in the browser.

The main acceptance scenario begins with Oat milk on Shelf A-02 at three units and Backroom B-04 at five. Selling two leaves one and five, total six. Moving four to the shelf leaves five and one, still six. Receiving twenty-four to the backroom leaves five and twenty-five, total thirty. The fourteen-day historical average remains six units per day because today's sale is excluded, so the supplier target is thirty and the post-sale suggestion is twenty-four.

Additional acceptance cases include same-token retries, insufficient selected-location stock, competing sales, stale counts, wrong-store QR payloads, price changes after a sale and image persistence across application restart. The requirements map identifies concrete tests and treats external hosting, real-camera optics and future classroom performance as separate verification obligations.

Chapter 3: Software Design and Architecture

3.1 Component architecture

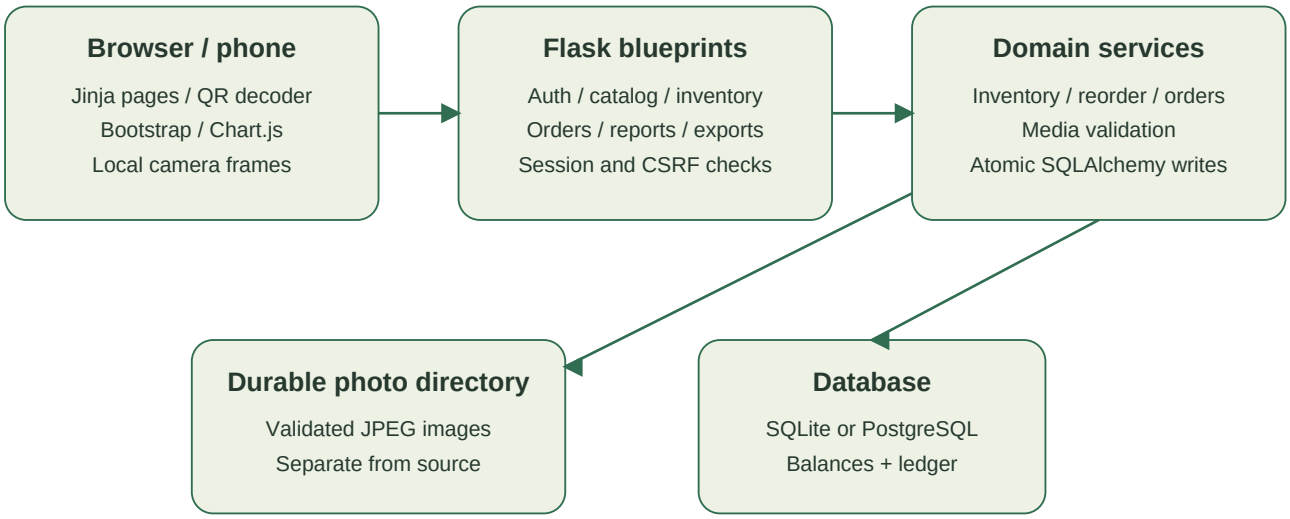


Figure 1. StockSense runtime architecture. The browser renders server-produced pages and runs a local QR decoder; Flask adapts requests to domain services; SQLAlchemy persists business records; normalized photos live in a configured persistent directory.

The application uses Flask's application factory pattern so tests can create isolated configurations and database connections [3]. Blueprints group authentication, catalog, inventory, orders, dashboard and exports. The HTTP adapter validates forms and invokes services. Business rules such as stock subtraction, duplicate-token handling and order receipt live in service modules rather than camera code.

Jinja2 supplies HTML templates. Bootstrap provides baseline controls and responsive layout; custom CSS gives the interface a consistent visual language. Chart.js renders daily units sold. Browser dependencies are served locally with retained license files, so the core demo does not depend on a CDN.

The data layer uses SQLAlchemy with SQLite locally and PostgreSQL for the hosting target [4]. Alembic migrations create the same schema from an empty database. The media service is separate because image failure must not affect inventory. The optional future AI adapter is absent from runtime; it is not a hidden dependency of reorder recommendations or stock operations.

This structure favors a comprehensible course project over additional service boundaries. It avoids a separate SPA build, reduces authentication complexity and allows the student to trace a stock action from form submission through one transaction to the visible ledger.

Chapter 3: Software Design and Architecture

3.2 Data model and integrity constraints

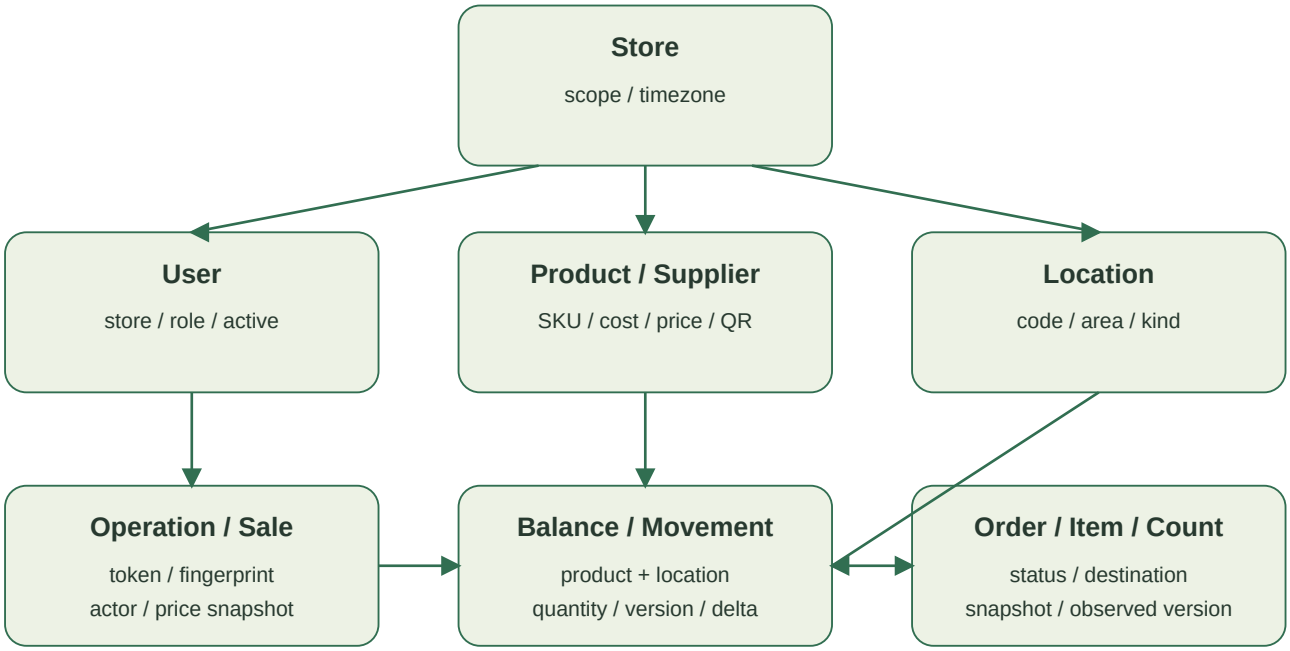


Figure 2. Core entity relationships. Store membership scopes users and business records. Product/location pairs identify stock balances; operation receipts link stock history to sales and transfers; purchase orders contain product lines with receiving destinations.

The principal entities are Store, User, Supplier, Product, Location, StockBalance, Operation, Sale, StockTransfer, StockMovement, ShelfRule, PurchaseOrder, PurchaseOrderItem and StockCountRequest. Composite foreign keys pair store identifiers with entity identifiers. A reference to a product or location from another store therefore fails even if an application-level lookup were accidentally omitted.

StockBalance is unique for each store/product/location tuple. Its quantity is nonnegative and its version increments with every mutation. Product total is calculated as the sum of balances; there is no second mutable product-level quantity to synchronize. The movement ledger records signed deltas and remains available through product history.

Operation supplies a unique store/token pair, request fingerprint and stored result. The fingerprint includes actor, action and normalized payload. The same token with matching fields returns the previous receipt; different fields are rejected. Sale records a price snapshot, and purchase-order lines record a unit-cost snapshot and destination. Changing catalog prices cannot rewrite historical revenue or order values.

Archives preserve referenced catalog records. Location archival is blocked while it has stock, pending counts, open order lines or shelf rules. Product archival requires resolving remaining stock and open orders. These policies favor an explainable audit trail over deleting records that historical transactions still reference.

Chapter 3: Software Design and Architecture

3.3 Transactions, concurrency and idempotency

Every mutation begins by updating the authenticated store's revision row. On PostgreSQL this obtains a row lock; on SQLite a write lock serializes writers. The service expires cached ORM state after acquiring the lock, then rechecks the active account and reads current records. This conservative design serializes writers within a small store, trading throughput for straightforward consistency.

A sale conditionally decrements the selected balance only if enough stock remains. It then inserts a Sale, StockMovement and Operation receipt in the same transaction. Any failure rolls back all writes. The nonnegative constraint and affected-row check provide additional protection beyond the store lock.

A transfer validates that source and destination are active, distinct and store-consistent. It subtracts the requested units from the source, adds the same units to the destination and inserts a transfer header. Both movements share the operation identifier. Failure in either leg restores the original state. Transfer movements never enter the sales average.

A count captures the observed balance version along with the physical quantity and reason. Owner approval compares that version with the current balance. An intervening sale, receipt, transfer or adjustment makes the count stale and requires a recount. A correction appends a signed compensating movement rather than rewriting old history.

Order receipt requires approved status and handles every line in one transaction. The status transition, all stock additions and the operation receipt commit together. An identical retry returns the stored result. A different receipt token after the order is received is rejected by lifecycle validation. These complementary checks address both network retries and repeated attempts through a newly loaded page.

Chapter 3: Software Design and Architecture

3.4 Time and exact reorder arithmetic

The store's IANA timezone defines business-calendar days. Sales timestamps are stored in UTC. The application converts local start and end dates into half-open UTC intervals, so reporting handles calendar boundaries and daylight-saving transitions without assuming every day contains twenty-four hours. A dedicated test checks a twenty-three-hour Chicago business day.

Reorder uses the previous fourteen completed local days and excludes the current day. For a product created inside that window, the denominator is the number of completed days since creation, capped at fourteen. Zero-sale days remain in the denominator. A product with no completed days displays insufficient history and uses its manual threshold.

The calculation is: average = sold / completed days; coverage = supplier lead time + safety days; forecast target = ceiling(average x coverage); target = maximum(manual threshold, forecast target); suggestion = maximum(0, target - total stock - approved incoming units). Python Fraction retains exact arithmetic before applying the ceiling. Rounded display averages do not drive the calculation.

For the proposal fixture, eighty-four units across fourteen days produces six per day. Three lead-time days plus two safety days gives five days' coverage. The forecast target is thirty. With eight units and no approved incoming order, the suggestion is twenty-two. An approved ten-unit order reduces it to twelve. Draft orders have no effect.

Days remaining is shown only when average sales is positive. Low stock means quantity at or below the manual threshold; forecast risk means estimated days remaining is at most coverage. An item at its threshold may correctly show a zero-unit order. The formula is explainable and deterministic; it does not model seasonality, lost demand during stockouts or learned forecasts.

Chapter 4: Implementation

4.1 Application setup and configuration

The repository contains the app factory, shared extensions, models, blueprint routes, domain services, templates, static assets, migration history, tests and documentation. pyproject.toml declares Python 3.12 and compatible dependency ranges; uv.lock records exact resolution. requirements.txt supplies pinned runtime dependencies for conventional hosting.

The application factory configures the database, upload directory, session cookies, request-size limit and CSRF support. Production configuration requires a random secret of adequate length and enables secure session cookies. Local runs persist a generated session secret in the ignored instance directory if the environment supplies none. The README recommends explicitly setting it in a private .env file.

Flask-Migrate/Alembic creates the initial schema. The health route performs a database query and returns a small JSON response. The seed command refuses to overwrite an existing store, reads passwords from the environment and creates only fictional records. A reconciliation command compares every location balance with its summed movement deltas.

The development seed contains eighteen products, three suppliers, three locations, thirty days of sales, original package illustrations, shelf configuration and example order states. It includes a second store for isolation exercises. A fixed reference date supports reproducible fixtures, while normal use anchors history to today's Chicago date.

GitHub Actions installs the locked environment and runs SQLite, PostgreSQL and Playwright checks. The proposed Render configuration installs runtime dependencies, migrates and starts Gunicorn, uses PostgreSQL and places media on an attached disk [6]. Configuration files are preparation artifacts; hosted execution and redeployment persistence require separate evidence. No secret or real business data belongs in the public repository.

Chapter 4: Implementation

4.2 Catalog, product information and photos

Owners create products with name, SKU, category, unit, supplier, cost, selling price, reorder level and a bounded description. Server validation converts decimal currency input to integer cents and rejects negative or over-precise prices. SKUs permit a restricted set of characters and are unique within a store. Template escaping renders description text as text rather than executable HTML.

New products start with zero stock unless the owner explicitly supplies an opening quantity and location. A positive opening balance is recorded through the stock service and produces a movement. Catalog editing never resets stock. The service rejects another opening operation once a product has movement history.

Photos are optional and handled separately after product save. The media service reads at most five megabytes, decodes the actual image, accepts JPEG/PNG/WebP, enforces a sixteen-megapixel limit and creates a bounded JPEG with a random storage key. EXIF orientation is applied before conversion and resizing. Replaced files are removed only after the database update succeeds. A neutral package illustration appears when no photo exists.

A photo is a human identification aid, not an image-recognition system. It never determines quantity or permission. An invalid upload cannot change inventory. Tests cover malformed content, oversized bytes, excessive decoded pixels, normalization dimensions and retrieval after constructing a new application instance with the same database and upload directory.

Supplier and location settings are owner-only. Locations include code, name, type, area, aisle, shelf and bin. The location view filters these fields and links to products held there. Staff management creates or disables staff accounts without exposing owner-role editing through that screen. Historical references remain intact when records are archived.

Chapter 4: Implementation

4.3 QR recognition and confirmation workflow

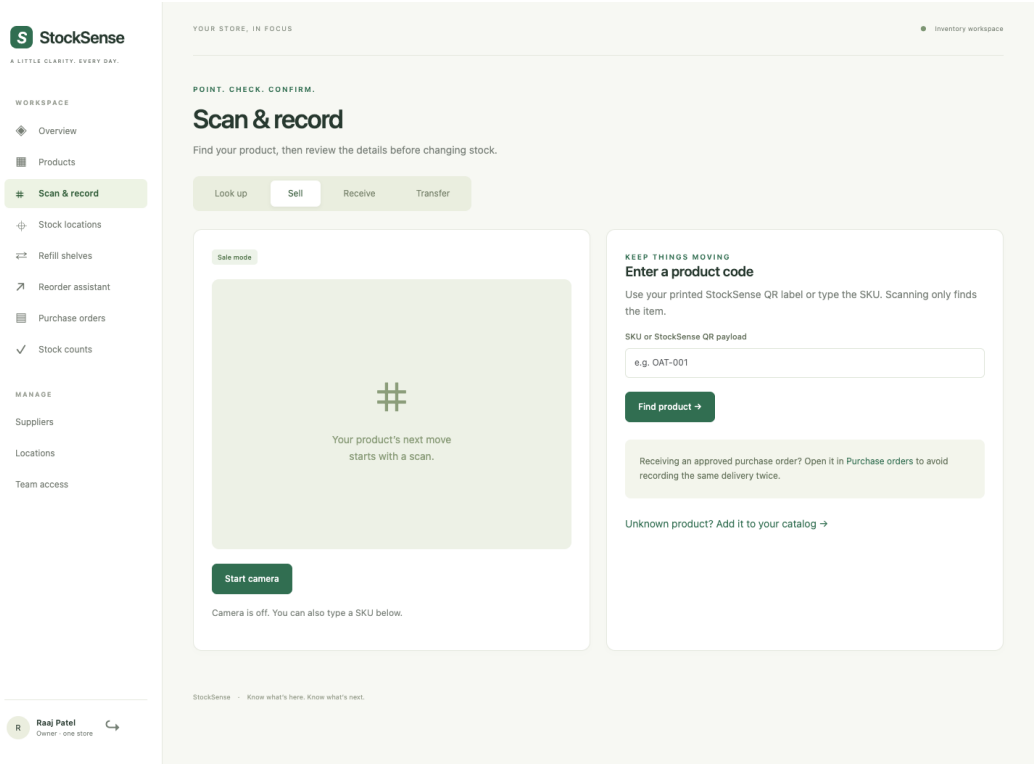


Figure 3. Actual Scan & record screen from the local application. Mode selection precedes camera use or manual input. The interface explains that finding a product does not itself change stock.

Each product receives a stable random QR token. Labels encode a versioned payload of the form stocksense:v1:product:token and include a readable name, SKU and unit. Changing catalog fields does not regenerate that token. The server resolves only active products in the signed-in store and rejects malformed or external URL payloads.

The scanner uses getUserMedia for video only, with no microphone permission [5]. A locally served jsQR decoder processes frames in the browser. Recognition stops the stream and opens the next step once; repeated frames do not add quantity or create transactions. Leaving the page or hiding it stops video tracks. If permission is denied or capture is unavailable, the manual SKU field remains available.

Sell, Receive and Transfer modes all lead to the same service-backed inventory forms. The user selects a positive quantity and physical locations. A review page shows projected before/after balances and product total. Only the confirmation POST invokes the mutation service. The server rechecks actual stock rather than trusting the preview.

For a purchase order, camera recognition only checks that the product belongs to an order line and returns to complete-order review. It cannot create a parallel standalone receipt. The user confirms all order quantities and destinations together. This keeps identification separate from the actual inventory transaction.

Chapter 4: Implementation

4.4 Location balances and shelf replenishment

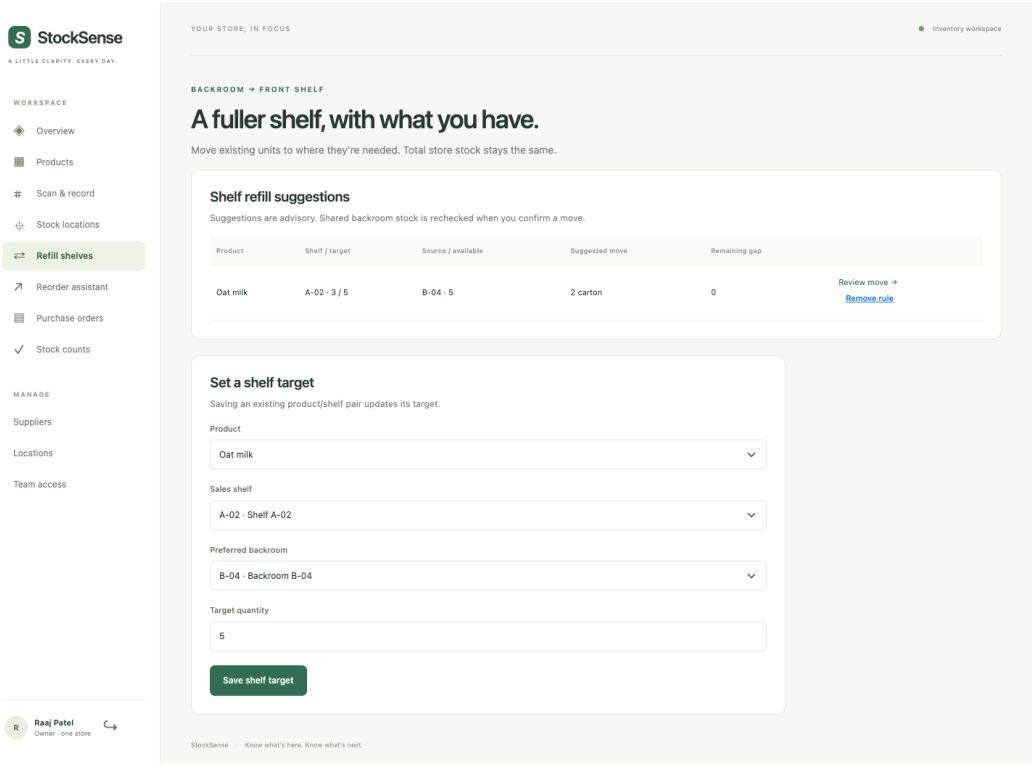


Figure 4. Actual shelf refill page. Suggestions show destination quantity and target, preferred source availability, proposed move and any remaining gap. The action opens review rather than moving stock immediately.

The stock service supports sale, standalone receipt, opening balance, transfer, order receipt and approved adjustment. Each operation records actor and reference information. Standalone deliveries require a supplier/delivery reference. A selected shelf cannot silently borrow from a backroom when the sale quantity is unavailable; the user must record an explicit transfer or choose the true source.

A shelf rule identifies a product, sales shelf, preferred backroom and target. The shelf gap is maximum(0, target - shelf quantity). The suggestion is the minimum of that gap and available preferred-source stock. If several shelves share a source, the list is advisory and does not reserve units. Execution rechecks current stock.

The demo begins at three shelf units and five backroom units. After a two-unit shelf sale, the shelf gap to target five is four. The suggested transfer moves four, producing shelf five and backroom one. Store total remains six. The transfer does not affect recorded sales, historical revenue or supplier reorder quantity.

This design turns location tracking into a useful operational action while preserving the purchasing authority of the store-level rule. A remaining shelf gap is not automatically converted into an order. Owners configure targets; both staff and owners can perform authorized transfers through the same transaction service.

Chapter 4: Implementation

4.5 Purchase orders and lifecycle controls

The reorder assistant groups selected positive suggestions by supplier and creates one draft per supplier. Draft lines contain product, quantity, unit-cost snapshot and receiving destination. Owners can change quantities or destinations and remove a line by setting its quantity to zero. A draft must retain at least one line unless it is cancelled.

The lifecycle is draft to approved to received, with cancellation allowed from draft or approved. Only drafts are editable. Approval is owner-only and re-evaluates current recommendations, including other approved incoming stock. If a saved quantity exceeds the current actionable suggestion, approval requires reducing it. This prevents two stale drafts from both purchasing the same apparent shortage.

Approval does not increase inventory. It only records incoming units for recommendation purposes. Complete receipt is available to staff and owners. The screen shows quantities and destinations and requires a checkbox confirming that every line was checked against the physical delivery. The service adds every line and transitions the order to received in one transaction.

Received or cancelled orders cannot be received. An identical operation retry returns its stored result without adding stock again. Tests inject a failure in a later line and verify that neither earlier additions nor status changes commit. Order CSV uses recorded cost snapshots, destination codes and line totals. Staff cannot access cost-based exports or approve/cancel an order by sending direct requests.

Partial receipt is intentionally excluded. The interface must not imply support for accepting some lines or changing received quantities. A delivery entered separately with an unrelated new token cannot always be recognized as the same physical event; references and clear order-specific receiving flows reduce that operational risk without overstating automatic duplicate detection.

Chapter 4: Implementation

4.6 Dashboard, reports and exports

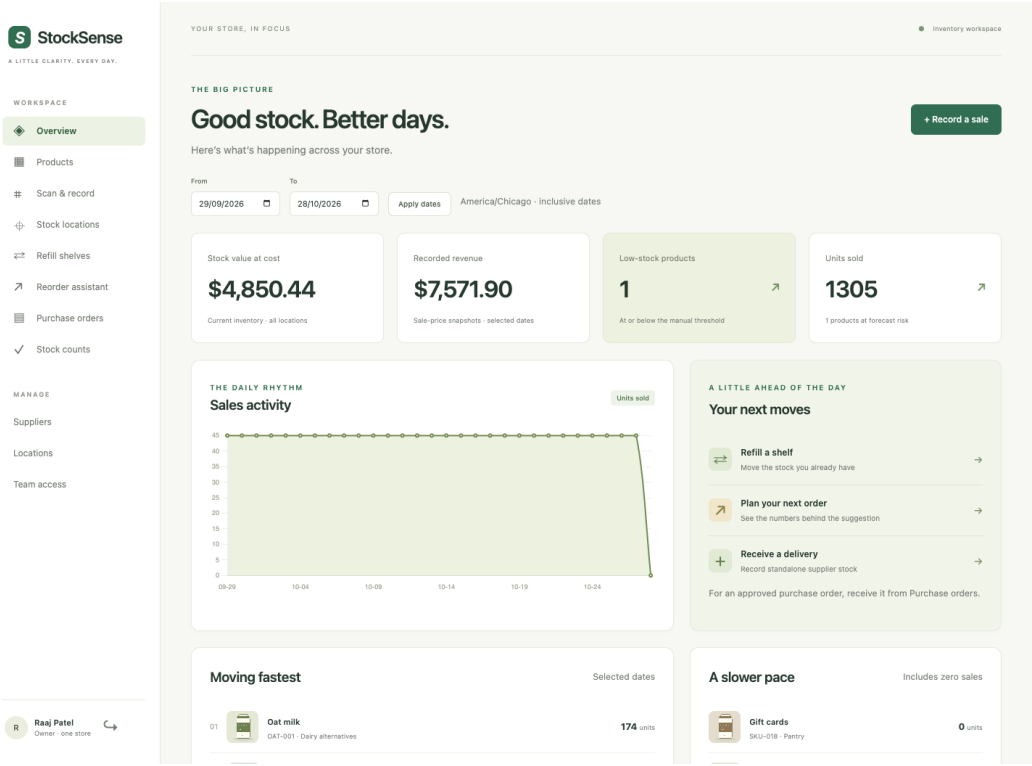


Figure 5. Actual owner overview. Cost valuation and historical revenue appear alongside units sold and stock warnings. The chart uses selected business-calendar dates, and next actions distinguish shelf refill from supplier purchasing.

Current stock value is the sum of quantity times current catalog cost across locations. Recorded revenue is the sum of historical sale quantities times their price snapshots within the selected date range. It is not profit. Staff see operational counts and units but do not receive owner-only cost metrics.

Fastest and slowest rankings use units sold, not revenue. Active products with no sales are retained in the slowest list. Date filters are inclusive in local business time and translated to UTC query bounds. Daily chart entries include zero-sales dates. An empty interval remains a readable report with zero totals.

CSV output covers inventory, dated sales, reorder suggestions and individual orders. Python's csv writer handles quoting, commas and newlines. User text that begins with a formula-trigger character after whitespace is prefixed so spreadsheet tools treat it as text. Costs use explicit cents columns rather than ambiguous floating-point values. All exports are owner-only and store-scoped.

The activity feed draws from movement history, showing action, location, signed quantity, actor and timestamp. Product detail exposes the latest hundred entries for that item. These views support explanation and troubleshooting without allowing a generic undo action that silently rewrites the past.

Chapter 5: Testing and Results

5.1 Test strategy and execution evidence

Testing follows the course's unit/integration/browser distinction. Pure arithmetic cases validate target calculations, zero-sales fallback, fractional averages and threshold boundaries. Integration tests exercise migrated databases, service transactions, store scoping, roles, media handling and reports. Playwright drives complete browser workflows through actual HTTP handlers and rendered pages.

Environment / suite	Tests	Failures	Errors	Result
SQLite service + HTTP	57	0	0	Passed
PostgreSQL service + HTTP	57	0	0	Passed
Playwright Chromium	5	0	0	Passed

Table 1. Actual automated execution summaries extracted from committed JUnit evidence. Counts refer to each suite invocation, not distinct requirements. The same service/HTTP suite runs independently on SQLite and PostgreSQL to expose database-specific behavior.

The fixture creates an empty schema through Alembic, two stores, owner/staff accounts, suppliers, products and initial location movements. Tests use isolated temporary SQLite databases or a dedicated disposable PostgreSQL database. No production or personal data is used. Each case can inspect the persisted result rather than relying only on a success message.

The browser fixture runs a local threaded HTTP server and a real browser. It covers sale, transfer, draft/approval/receipt, catalog setup, photo upload, staff settings, mobile overflow and camera-denial fallback. A generated video stream contains an actual product QR image, allowing the real decoder to locate and parse it. This verifies software decoding behavior while explicitly excluding physical camera optics.

Test commands and limitations are preserved in docs/test-results.md. A configured GitHub workflow alone is not reported as a passed hosted run. The release checklist records hosted CI, public access and deployment checks using their own evidence rather than inferring them from local execution.

Chapter 5: Testing and Results

5.2 Transaction and arithmetic outcomes

The core reorder fixture sells eighty-four units in the previous fourteen completed days. Its expected average is six, target thirty and suggestion twenty-two from stock eight. The test confirms those values and verifies that a sale recorded today does not enter the historical numerator. A newly created product uses its available completed days; a zero-history case avoids division by zero and uses the threshold.

The demo stock test confirms the sequence eight to six to six to thirty. It checks both location balances after each step and runs ledger reconciliation. Retrying the same sale token creates exactly one sale. Reusing the token with changed quantity is rejected. Transfers preserve total units and supplier recommendation inputs.

Two concurrent sales compete for a shelf with three units, each requesting two. Exactly one succeeds and the remaining quantity is one. Two concurrent four-unit transfers compete for a backroom holding five; only one succeeds, leaving one there. The final balances reconcile and never become negative. These cases execute independently on SQLite and PostgreSQL.

Failure injection tests stop the second transfer leg or a later purchase-order receipt line. The expected result is complete rollback: original balances, original order state and no partial ledger effect. The tests confirm that behavior. Count tests record a physical observation, perform an intervening sale, then reject stale approval. A fresh count succeeds and reconciles.

A snapshot test records a sale, changes the current product price and confirms historical revenue remains unchanged. Calendar tests include an empty date interval and the Chicago spring-forward boundary. CSV cases cover formula-leading text. Image cases reject malformed, oversized and high-pixel inputs. These checks focus on meaningful failure modes rather than a coverage percentage that could conceal missing business rules.

Chapter 5: Testing and Results

5.3 Interface verification and observed defect

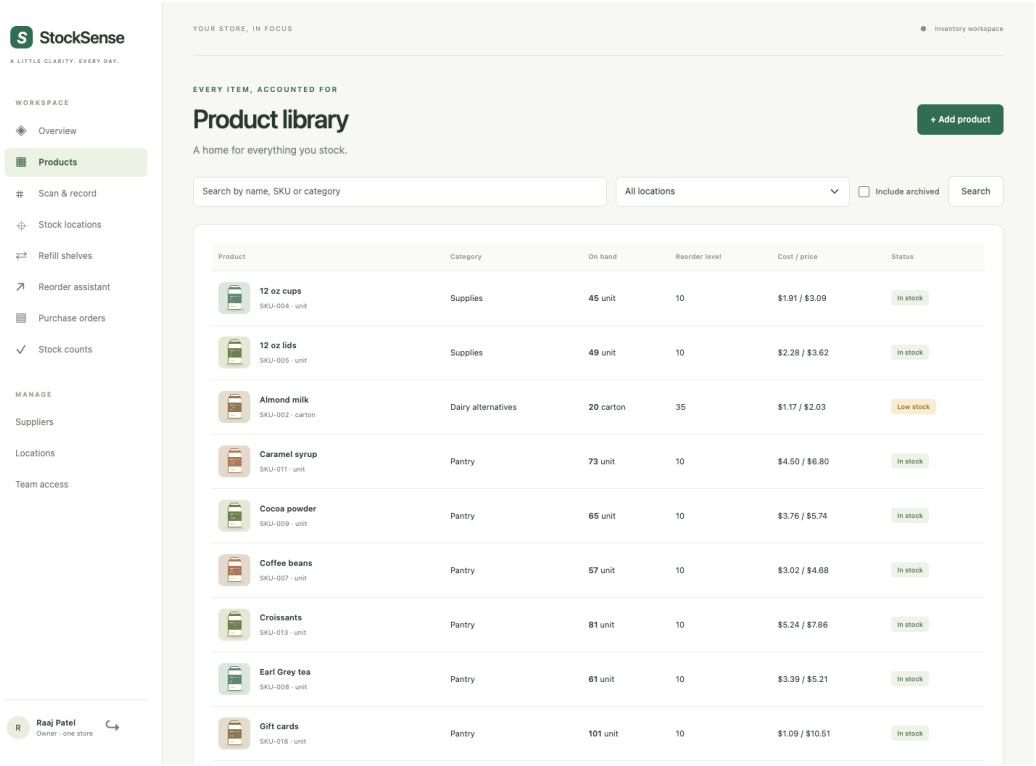


Figure 6. Actual product library. Photos, SKUs, units, category, quantity, threshold and status support product selection. Owner-only cost/price information appears in the owner session; staff rendering is checked separately.

The interface was captured from the running application using Playwright. Screenshot inspection identified a real defect: the sales chart container was blank because the shared application script executed before the deferred Chart.js script. The absence of a thrown exception made ordinary page rendering an insufficient test. Initialization was moved to DOMContentLoaded, and the browser login helper now asserts that Chart.getChart returns an initialized chart.

Mobile checks render the main operational pages at a 390-pixel viewport and assert that document width does not overflow. Tables use scrollable containers. The owner setup workflow creates a supplier, a location, a product with opening stock, a photo and a staff account, then verifies the printed label. This extends coverage beyond merely navigating pre-seeded pages.

Camera denial is simulated through the browser API and must leave manual entry usable. Synthetic camera video verifies real QR decoding and confirms recognition does not deduct stock. These tests do not establish camera focus, lighting quality, device-specific permission prompts or performance in the classroom. The demo script includes a separate physical rehearsal checklist for Raaj.

Chapter 5: Testing and Results

5.4 Executability, release and remaining checks

The root README documents prerequisites, locked installation, private environment setup, migration, seeding, launch commands, safe account names, interface behavior and testing. The seed requires user-supplied passwords and refuses to overwrite an existing store. Local uploads and secrets are ignored by Git. Runtime assets and their licenses are included in the source distribution.

A fresh tracked-source rehearsal passed in a separate temporary directory: locked installation, empty-database migration, schema comparison, seed, reconciliation, and a two-worker Gunicorn process serving the health and login endpoints. PostgreSQL compatibility is supported by actual local test execution, while hosted proxy behavior and persistent storage remain separate checks. The Render blueprint describes a PostgreSQL database and durable media disk rather than writing uploads to the disposable source directory [6].

The public repository was published at https://github.com/Raajp10/stocksense and verified through unauthenticated repository-page and raw README requests. This direct URL is included here for the final BrightSpace submission [1]. Hosted GitHub Actions run 40745866259 also passed for the published implementation, including SQLite, PostgreSQL, browser workflows, migration, seeding and reconciliation. Subsequent release runs remain visible under the repository Actions tab.

The remaining physical and scheduled obligations belong to the student: confirm the announced demonstration time, rehearse on the intended camera/browser, understand the transaction and reorder code, present on December 9 and submit the report/repository URL by December 13 at 11:59 p.m. The source PDF does not specify an LMS timezone, so that detail requires confirmation.

No measured shop outcomes are reported. The application's evidence supports defined software behaviors using synthetic data. A real adoption study, larger-volume performance analysis, deployment restore exercise and physical-camera rehearsal would provide additional confidence before operational use beyond the classroom.

Chapter 6: Conclusion, Limitations, and Future Improvements

StockSense implements a location-aware inventory workflow with explicit transactions and explainable replenishment. Owners and staff can identify a product, inspect its physical quantities and record an authorized action. Stable QR labels and photos make selection easier, while manual input provides a fallback. The ledger, request token and count version preserve a traceable relationship between intended action and saved state.

The project demonstrates several software design principles: separation of HTTP adapters from domain services, a single source of stock truth, database-enforced relationships, atomic multi-record updates, historical snapshots and evidence-driven testing. The reorder formula is small enough to explain in class and includes visible inputs rather than concealing its assumptions.

The system has deliberate limits. Whole-number units exclude weighed ingredients and conversion. Full receipt excludes partial deliveries. The catalog does not manage lots, expiry, returns or recipes. Sales do not process payments. Recorded sales can understate demand during stockouts. Store-level writer serialization is appropriate for a small shop but would need performance evaluation before higher-volume use. New-token duplicate physical deliveries still require human discipline.

Future improvements can follow the existing boundaries: previewed CSV catalog import, manufacturer barcode mapping, partial-receipt state design, explicit returns/revenue reversals, durable object storage, stronger operational monitoring and a tested read-only AI adapter. Any learned forecasting would require appropriate data and evaluation rather than replacing an explainable formula with an unsupported claim.

The current handoff is the local prototype MVP requested by Raaj in response to the hosting question. A one-command launcher prepares and runs the application, and paid hosting is deferred. The public source and automated tests support evaluation. Physical-camera rehearsal, student defense and the scheduled course submission remain personal follow-up obligations. The prepared Render configuration supports a later deployment once account and cost choices are accepted.

References

[1] O. T. Ajayi, "Software Design Course Project: Course Project Submission Guidelines," Software Design I, Purdue University Northwest, Fall 2026. User-supplied PDF, three pages.

[2] R. Patel, "StockSense: Smart inventory and reorder assistant for small businesses," approved project proposal, Software Design Practicum, Oct. 19, 2026. User-supplied presentation, twelve slides.

[3] Pallets Projects, "Flask documentation: Installation and application factories." Available: https://flask.palletsprojects.com/en/stable/installation/ and https://flask.palletsprojects.com/en/stable/patterns/appfactories/ . Accessed Oct. 28, 2026.

[4] SQLAlchemy authors, "SQLAlchemy 2.0 documentation." Available: https://docs.sqlalchemy.org/en/20/ . Accessed Oct. 28, 2026.

[5] MDN contributors, "MediaDevices: getUserMedia() method." Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia . Accessed Oct. 28, 2026.

[6] Render, "Deploy a Flask App," "Blueprint YAML Reference," "Persistent Disks," and "Deploying on Render." Available: https://render.com/docs/deploy-flask , https://render.com/docs/blueprint-spec , https://render.com/docs/disks , https://render.com/docs/deploys . Accessed Oct. 28, 2026.

[7] C. Wolfe and contributors, "jsQR." Available: https://github.com/cozmo/jsQR . Apache License 2.0. QR generator: Lincoln Loop and contributors, "qrcode," https://pypi.org/project/qrcode/ . Accessed Oct. 28, 2026.

[8] Microsoft, "Playwright for Python documentation." Available: https://playwright.dev/python/ . pytest contributors, "pytest documentation," https://docs.pytest.org/ . Accessed Oct. 28, 2026.

Third-party asset licenses are retained under app/static/vendor and listed in THIRD_PARTY_NOTICES.md. Original geometric product fixtures are generated by app/cli.py. Significant AI assistance is disclosed in docs/ai-usage.md. Screenshots were captured from the local running application and are not design mockups.